

Kiste v0.9.4

Key Management Code Design

SecretRef, KeyRef, BackendRef, and TrustPolicy

Item	Description
Status	Draft engineering document
Target repository	KisteBox/kiste-py
Target version	0.9.4
Release theme	Key Management Integration Starting
Core rule	Kiste manages references, not secrets.

1. Purpose

Kiste v0.9.4 introduces the first key-management integration layer. It is not a password manager, Vault replacement, or IAM engine. It is a trust gateway for safe deployment and monitoring.

Kiste manages references, not secrets.

Kiste should detect secret usage, model safe references, validate backends, redact plans, and block unsafe deployment.

2. Scope and Non-goals

In scope

- Secret discovery and plaintext secret detection.
- SecretRef, KeyRef, BackendRef, and TrustPolicy models.
- Backend validation for SOPS first, with Vault and cloud KMS reference placeholders.
- Plan redaction and deploy blocking when secret strategy is missing.
- Safe export to GitOps, Kubernetes placeholders, and External Secrets compatible manifests.

Non-goals

- No raw secret storage inside Kiste.
- No secret reveal or print command.
- No full Vault replacement.
- No full IAM permission engine.
- No automatic production key rotation in v0.9.4.
- No production deploy with unresolved SecretRef or placeholder secrets.

3. New Core Objects

Item	Description
SecretRef	Pointer to where a secret lives. It never contains the secret value.
KeyRef	Pointer to the key identity used by a backend for encryption, decryption, or access.
BackendRef	Description of a configured secret or key backend such as SOPS, Vault, or cloud KMS.
TrustPolicy	Rules that control secret safety, redaction, deploy gates, and backend requirements.

SecretRef YAML

```
kind: SecretRef
name: DATABASE_URL
backend: sops
path: secrets/dev.enc.yaml
key: DATABASE_URL
environment: dev
required_for:
  - backend
```

KeyRef YAML

```
kind: KeyRef
name: dev-age-key
backend: age
identity_ref: env:SOPS_AGE_KEY_FILE
usage:
```

```
- decrypt-sops
environment: dev
```

BackendRef YAML

```
kind: BackendRef
name: dev-sops
type: sops
status: configured
files:
  - secrets/dev.enc.yaml
```

TrustPolicy YAML

```
kind: TrustPolicy
name: default-secret-policy
rules:
  no_plaintext_secrets: true
  no_secret_values_in_plan: true
  require_secret_refs_for_deploy: true
  require_backend_for_secret_refs: true
```

4. Recommended Code Layout

```
src/
kiste/
  secrets.py
  key_management.py

kiste_core/
  key_management/
    __init__.py
    models.py
    config.py
    scanner.py
    validators.py
    planner.py
    redaction.py
    backends.py
    sops.py
    vault.py
    kms.py
    external_secrets.py
    output.py
    doctor.py
```

5. Data Model Skeleton

```
from dataclasses import dataclass, field
from typing import Any

@dataclass
class SecretRef:
    name: str
    backend: str
    key: str
    kind: str = "SecretRef"
    path: str | None = None
```

```

environment: str | None = None
required_for: list[str] = field(default_factory=list)
metadata: dict[str, Any] = field(default_factory=dict)

```

```

@dataclass
class KeyRef:
    name: str
    backend: str
    identity_ref: str
    kind: str = "KeyRef"
    usage: list[str] = field(default_factory=list)
    environment: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)

```

```

@dataclass
class BackendRef:
    name: str
    type: str
    kind: str = "BackendRef"
    enabled: bool = True
    files: list[str] = field(default_factory=list)
    address_ref: str | None = None
    key_refs: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)

```

```

@dataclass
class TrustPolicy:
    name: str = "default"
    no_plaintext_secrets: bool = True
    no_secret_values_in_git: bool = True
    no_secret_values_in_plan: bool = True
    require_secret_refs_for_deploy: bool = True
    require_backend_for_secret_refs: bool = True
    redact_secret_names: bool = False

```

6. Public Python API

```

import kiste

kiste.secrets.scan(path=".")
kiste.secrets.validate(path=".")
kiste.secrets.backend_list(path=".")
kiste.secrets.backend_set("sops", path=".")
kiste.secrets.ref_add(
    name="DATABASE_URL",
    backend="sops",
    path="secrets/dev.enc.yaml",
    key="DATABASE_URL",
    environment="dev",
)
kiste.secrets.ref_list(path=".")
kiste.secrets.plan(target="kubernetes", mode="gitops")

```

Lifecycle integration

```
read = kiste.read(".", groups=["secrets"])
inspection = kiste.inspect(read, groups=["secrets", "key-management"])
plan = kiste.plan(inspection, target="kubernetes", mode="gitops")
review = kiste.review(plan, decision="approve")
deployment = kiste.deploy(review)
monitor = kiste.monitor(groups=["secrets"], once=True)
```

7. CLI Contract

Use the existing secrets subsystem. Do not create a new top-level key command in v0.9.4.

```
kiste secrets scan
kiste secrets doctor
kiste secrets backend list
kiste secrets backend set sops
kiste secrets backend inspect sops

kiste secrets ref add DATABASE_URL --backend sops --path secrets/dev.enc.yaml --key DATABASE_URL --environment dev

kiste secrets ref list
kiste secrets ref inspect DATABASE_URL
kiste secrets plan
kiste secrets validate
```

Commands to avoid

```
kiste secrets print DATABASE_URL
kiste secrets reveal DATABASE_URL
```

8. YAML Contract

```
kiste: "0.9.4"
schema: "kiste.workspace/v0.9.4"
kind: Workspace

workspace:
  name: ai-product-platform
  owner: group:default/platform-team
  lifecycle: experimental

key_management:
  enabled: true
  strategy: external
  default_backend: sops

backends:
  sops:
    enabled: true
  files:
    - secrets/dev.enc.yaml
  key_refs:
    - dev-age-key

vault:
  enabled: false
  address_ref: env:VAULT_ADDR
```

```

aws_kms:
  enabled: false
  key_refs: []

rules:
  no_plaintext_secrets: true
  no_secret_values_in_git: true
  no_secret_values_in_plan: true
  require_secret_refs_for_deploy: true
  require_backend_for_secret_refs: true

secrets:
  refs:
    - name: DATABASE_URL
      kind: SecretRef
      backend: sops
      path: secrets/dev.enc.yaml
      key: DATABASE_URL
      environment: dev
      required_for:
        - backend

keys:
  refs:
    - name: dev-age-key
      kind: KeyRef
      backend: age
      identity_ref: env:SOPS_AGE_KEY_FILE
      usage:
        - decrypt-sops
      environment: dev

deploy:
  require_approved_plan: true
  require_secret_strategy: true
  allow_secret_placeholders:
    dev: true
    staging: false
    prod: false

```

9. Schema IDs

```

kiste.config/v0.9.4
kiste.workspace/v0.9.4
kiste.secret-ref/v0.9.4
kiste.key-ref/v0.9.4
kiste.backend-ref/v0.9.4
kiste.trust-policy/v0.9.4
kiste.secrets.scan/v0.9.4
kiste.secrets.plan/v0.9.4
kiste.secrets.validation/v0.9.4

```

10. Output Layout

```

.kiste/
  key-management/
    backends.json

```

```
key-refs.json
secret-refs.json
trust-policy.json
validation-report.json
rotation-report.json

secrets/
scan-report.json
plaintext-findings.json
ref-report.json
secret-plan.json
redaction-report.json

plans/
secret-wiring-plan.json

exports/
external-secrets/
sops/
kubernetes/
```

11. Doctor Groups and Checks

New groups

```
secrets
key-management
sops
vault
kms
external-secrets
```

Recommended checks

```
secrets.no_plaintext_values
secrets.refs.declared
secrets.refs.no_raw_values
secrets.plan.redacted
secrets.gitignore.env_files

key_management.backend.configured
key_management.key_refs.valid
key_management.default_backend.exists
key_management.deploy_strategy.declared

sops.files.encrypted
sops.key_ref.declared

vault.address_ref.declared
vault.no_token_value_in_config

kms.key_refs.valid_shape

external_secrets.manifests.safe
external_secrets.secret_store_declared

deploy.secret_strategy.required
deploy.secret_placeholders.blocked_for_prod
```

12. Scanner Behavior

- Scan repository files for secret-looking values without printing or storing the values.
- Detect .env, .env.example, Kubernetes Secret manifests, SOPS files, Vault references, ExternalSecret manifests, GitHub Actions secret usage, Docker Compose environment usage, Helm values, Kustomize secretGenerator, and cloud KMS references.
- Store redacted findings only.
- Treat test fixtures and examples separately when possible, but still warn if values look real.

13. Redaction Rules

```
def redact_secret_value(value: str) -> str:
    if not value:
        return ""
    return "<redacted>"

def redact_finding(finding: dict) -> dict:
    clean = dict(finding)
    if "value" in clean:
        clean["value"] = "<redacted>"
    return clean
```

- Plans must not include raw secret values.
- Review reports must not include raw secret values.
- Logs must not include raw secret values.
- Secret names may be shown by default, unless redact_secret_names is true.

14. Planning Behavior

- Generate a secret wiring plan from detected environment needs and declared SecretRefs.
- Prefer External Secrets output when configured.
- Allow Kubernetes placeholder output for development only.
- Block production deploy if placeholder secrets are unresolved.

Kubernetes placeholder example

```
apiVersion: v1
kind: Secret
metadata:
  name: backend-secrets
  annotations:
    kiste.io/placeholder: "true"
type: Opaque
stringData:
  DATABASE_URL: "__KISTE_SECRET_REF_DATABASE_URL__"
```

External Secrets example

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: backend-secrets
spec:
  secretStoreRef:
    name: app-secret-store
    kind: SecretStore
  target:
    name: backend-secrets
```



```
data:
- secretKey: DATABASE_URL
  remoteRef:
    key: backend/dev
  property: DATABASE_URL
```

15. Deploy Safety Gates

- Deploy must require an approved plan.
- Deploy must require a declared secret strategy when secrets are required.
- Deploy must fail if plans contain raw secret values.
- Deploy must fail if production contains unresolved placeholders.
- Deploy must fail if SecretRefs point to missing backends.

Allowed:
`kiste deploy --approved-plan .kiste/reviews/latest.approved.json`

Avoid:
`kiste deploy`

16. Monitor Behavior

- Detect new plaintext secret findings after deployment.
- Detect removed or broken secret backend configuration.
- Detect SOPS files that are no longer encrypted.
- Detect broken ExternalSecret or SecretStore manifests when runtime access is available.
- Emit rotation warnings where metadata exists.

```
kiste monitor --group secrets --once
kiste monitor --group secrets --watch
kiste monitor --group key-management --interval 60
```

17. Implementation Order

1. Add `kiste_core/key_management/models.py`.
2. Add YAML loader support for `key_management`, `secrets.refs`, and `keys.refs`.
3. Add secrets scanner.
4. Add plaintext finding detector.
5. Add `SecretRef` and `KeyRef` validation.
6. Add backend registry.
7. Add SOPS backend metadata support.
8. Add redaction utility.
9. Add secrets plan output.
10. Add CLI commands.
11. Add doctor groups.
12. Add deploy safety gate.
13. Add monitor checks.
14. Add tests.

18. Acceptance Tests

15. `kiste --version` returns 0.9.4.
16. `kiste config schema` returns `kiste.config/v0.9.4`.
17. `kiste secrets scan` writes `.kiste/secrets/scan-report.json`.
18. Plaintext secret detection works on fixtures.
19. `SecretRef` can be added, listed, and inspected.

20. KeyRef can be declared and validated.
21. Backend set/list/inspect works for SOPS.
22. Plans never contain raw secret values.
23. Deploy fails without approved plan.
24. Deploy fails without secret strategy when secrets are required.
25. Production deploy rejects unresolved placeholders.
26. Monitor detects changed plaintext secret findings.
27. Doctor supports secrets and key-management groups.
28. Old secrets commands remain compatible or warn clearly.

19. Final Engineering Rule

Kiste v0.9.4 makes deployment trust-aware.

Before Kiste deploys anything, it must know:

- where secrets live,
- which backend owns them,
- which key reference protects them,
- whether the plan is redacted,
- and whether the user approved the secret strategy.